DSA

1. Longest Subarray with Sum K Find the maximum length contiguous subarray with sum K; values may be negative

Code:

```
class Solution {
    public int longestSubarray(int[] arr, int k) {
        HashMap<Integer, Integer> map = new HashMap<>();

        int sum = 0;
        int maxLen = 0;

        for (int i = 0; i < arr.length; i++) {
            sum += arr[i];
            if (sum == k) {
                maxLen = i + 1;
            }
            if (map.containsKey(sum - k)) {
                int len = i - map.get(sum - k);
                maxLen = Math.max(maxLen, len);
            }
            if (!map.containsKey(sum)) {
                map.put(sum, i);
            }
        }
        return maxLen;
    }
}
```

2. Minimum Window Containing All Required Skills Return the smallest substring of S containing all characters of T with required frequencies.

Code:

```
class Solution {
    public String minWindow(String s, String t) {
        if(t.length()>s.length()) return "";
        Map<Character,Integer> tf = new HashMap<>();
        for(char ch : t.toCharArray())
            tf.put(ch,tf.getOrDefault(ch,0)+1);
        Map<Character,Integer> sf = new HashMap<>();
        int l=0,minLen = Integer.MAX_VALUE,index=0, count=0;
        for(int r=0;r<s.length();r++){
            char chR = s.charAt(r);
            sf.put(chR,sf.getOrDefault(chR,0)+1);
            if(tf.containsKey(chR)&&sf.get(chR).intValue()==tf.get(chR).intValue()
)count++;
```

```
            while(count==tf.size()){
                if(minLen>r-l+1){
                    minLen=r-l+1;
                    index=l;
                }
                char chL = s.charAt(l);
                sf.put(chL,sf.get(chL)-1);
                l++;
                if(tf.containsKey(chL)&&sf.get(chL)<tf.get(chL)){
                    count--;
                    break;
                }
            }
        }
        return (minLen==Integer.MAX_VALUE)?"":s.substring(index,index+minLen);

    }
}
```

3. Merge Overlapping Intervals Merge overlapping meeting intervals and return the resulting non-overlapping schedule.

Code:

4. Kth Largest Element in a Stream After each insertion, output the kth largest value once at least k values exist

Code:

```
class KthLargest {
    private PriorityQueue<Integer> minHeap;
    private int k;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        minHeap = new PriorityQueue<>();

        for (int num : nums) {
            add(num);
        }
    }

    public int add(int val) {
        minHeap.offer(val);

        if (minHeap.size() > k) {
            minHeap.poll();
```

```
        }

        return minHeap.peek();
    }
}
```

5. Detect and Remove Cycle in Linked List Detect a cycle and remove it without losing any nodes

Code:

```
class Solution {
    public static void removeLoop(Node head) {
        Node p1 = head;
        Node p2 = head;
        while(p2!=null&&p2.next!=null){
            p2=p2.next.next;
            p1=p1.next;
            if(p1==p2){
                if(p1==head && p2==head){
                    while(p1.next!=head){
                        p1=p1.next;
                    }
                    p1.next=null;
                    return;
                }
                p1=head;
                while(p1.next!=p2.next){
                    p1=p1.next;
                    p2=p2.next;
                }
                p2.next=null;
                return;
            }
        }

    }
}
```

6. Lowest Common Ancestor in Binary Tree Return the lowest common ancestor of two existing nodes in a binary tree

Code:

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root==null) return null;
        if(root==p||root==q)
            return root;
        TreeNode left = lowestCommonAncestor(root.left,p,q);
```

```
        TreeNode right = lowestCommonAncestor(root.right,p,q);
        if(left !=null && right !=null) return root;
        if(left==null) return right;
        return left;
    }
}
```

7. Shortest Path in a Binary Matrix Find shortest top-left to bottom-right path through open cells using 8 directions

Code:

```
class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        int n = grid.length;
        if(grid[0][0]==1||grid[n-1][n-1]==1) return -1;
        if (n == 1) return 1;
        Queue<int[]> q = new LinkedList<>();
        q.add(new int[]{0,0});
        grid[0][0]=1;
        int[][] dir = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
        int dis =1;
        while(!q.isEmpty()){
            int size = q.size();
            for(int i =0;i<size;i++){
                int[] curr = q.poll();
                int r = curr[0],c=curr[1];
                for(int[] d:dir){
                    int nr = r+d[0],nc = c+d[1];
                    if(nr==n-1&&nc==n-1) return dis+1;
                    if(nr<0||nr>=n||nc<0||nc>=n||grid[nr][nc]==1) continue;
                    grid[nr][nc]=1;
                    q.add(new int[]{nr,nc});
                }
            }
                dis++;
        }
        return -1;
    }
}
```

8. Course Completion Order Return one valid course order from prerequisite pairs, or empty if impossible

Code:

```
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        int[] res = new int[numCourses];
        int k=0;
```

```
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) {
            graph.add(new ArrayList<>());
        }

        int[] indegree = new int[numCourses];

        for (int[] pre : prerequisites) {
            int course = pre[0];
            int prereq = pre[1];

            graph.get(prereq).add(course);
            indegree[course]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) {
            if (indegree[i] == 0) {
                queue.add(i);
            }
        }

        int count = 0;

        while (!queue.isEmpty()) {
            int current = queue.poll();
            res[k++]=current;

            for (int neighbor : graph.get(current)) {
                indegree[neighbor]--;
                if (indegree[neighbor] == 0) {
                    queue.add(neighbor);
                }
            }
        }
        if(k!=numCourses) return new int[]{};
        return res;
    }
}
```

9. Minimum Coins for Amount Find the minimum coins required for a target amount, or -1 if impossible

Code:

```
class Solution {

    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
```

```
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }

        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

10. Range Maximum Query with Updates Support point updates and maximum queries over an inclusive range [L,R].

Code:

```
class SegmentTree {
    int[] tree;
    int n;

    SegmentTree(int[] arr) {
        n = arr.length;
        tree = new int[4 * n];
        build(arr, 0, 0, n - 1);
    }

    void build(int[] arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
            return;
        }

        int mid = (start + end) / 2;

        build(arr, 2 * node + 1, start, mid);
        build(arr, 2 * node + 2, mid + 1, end);

        tree[node] = Math.max(
            tree[2 * node + 1],
            tree[2 * node + 2]
        );
    }

    void update(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = val;
```

```
        return;
    }

    int mid = (start + end) / 2;

    if (idx <= mid) {
        update(2 * node + 1, start, mid, idx, val);
    } else {
        update(2 * node + 2, mid + 1, end, idx, val);
    }

    tree[node] = Math.max(
        tree[2 * node + 1],
        tree[2 * node + 2]
    );
}

int query(int node, int start, int end, int L, int R) {
    if (R < start || end < L) {
        return Integer.MIN_VALUE;
    }

    if (L <= start && end <= R) {
        return tree[node];
    }

    int mid = (start + end) / 2;

    int leftMax = query(2 * node + 1, start, mid, L, R);
    int rightMax = query(2 * node + 2, mid + 1, end, L, R);

    return Math.max(leftMax, rightMax);
}

void update(int idx, int val) {
    update(0, 0, n - 1, idx, val);
}

int rangeMax(int L, int R) {
    return query(0, 0, n - 1, L, R);
}

public static void main(String[] args) {
    int[] arr = {2, 5, 1, 4, 9, 3};

    SegmentTree st = new SegmentTree(arr);

    System.out.println(st.rangeMax(1, 4)); // 9

    st.update(2, 10);
```

```
        System.out.println(st.rangeMax(1, 4)); // 10
        System.out.println(st.rangeMax(0, 2)); // 10
    }
}
```